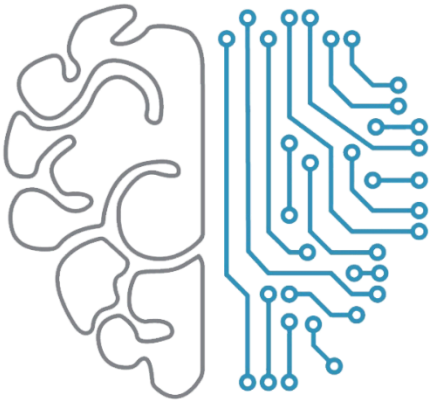
LAPORAN TEORI
STRUKTUR DATA



NAMA : ZAIN AKBAR RIZKIA

NIM : 202531091

KELAS : E

DOSEN : Rahma Farahningrum, S.Kom., M.Kom.

NO. PC : 4

ASISTEN : 1. Todo Christanto Pangamudi Simanjuntak

: 2. Harits Nakhlah Putra

: 3. Muhammad Fadel Desyaputra

: 4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026

BAB I

LAPORAN TEORI

1.1 Delete Kiri, Delete Kanan, dan Delete Tengah

Di single linked list, tiap node cuma punya pointer `next` (Wikipedia, 2026a). Jadi sebelum node dibuang, pointer di sekitarnya harus disambung ulang dulu, kalau tidak sisa list-nya hilang. Pointer `awal` (head) dan `akhir` (tail) juga harus ikut diperbarui.

Delete kiri (`hapus_awal`) membuang node paling depan. Alurnya: cek `awal == NULL` (list kosong, berhenti), simpan node depan ke `hapus`, geser `awal = awal->next`, lalu `delete hapus` (cppreference.com, 2026). Kalau list cuma punya satu node, `akhir` juga jadi `NULL`. Tidak ada traversal, jadi cost-nya $O(1)$.

Delete kanan (`hapus_akhir`) membuang node paling belakang. Pointer `akhir` menunjuk node terakhir, tapi tidak ada pointer ke node sebelumnya. Jadi list ditelusuri dari `awal` sampai ketemu `b1` dengan `b1->next == akhir`, lalu `b1->next = NULL` dan `akhir = b1`. Karena ada traversal, cost-nya $O(n)$.

Delete tengah (`hapus_tengah`) membuang node berisi nilai `x`. Cari node sebelumnya (`b`), lalu lewati target dengan `b->next = b2` (`b2` = node sesudah target). Kalau `x` ada di ujung, fungsi tinggal memanggil delete kiri/kanan karena `awal`/`akhir` ikut berubah. Kalau `x` tidak ketemu, tampilkan pesan tidak valid.

```
91  void hapus_awal()
92  {
93      if(awal==NULL)
94      {
95          cout<<"Maaf....List Kosong!!!\n";
96      }
97      else if (awal==akhir)
98      {
99       hapus=awal;
100      awal=NULL;
101      akhir=NULL;
102     }
103     else
104     {
105        hapus=awal;
106        awal=awal->next;
107     }
108 }
```

Gambar 1.1 `hapus_awal`

```
110 void hapus_akhir()
111 {
112     if(awal==NULL)
113     {
114         cout<<"Maaf....List Kosong!!!\n";
115     }
116     else if(awal==akhir)
117     {
118     hapus=awal;
119     awal=NULL;
120     akhir=NULL;
121     }
122     else
123     {
124         b1=awal;
125       while(b1->next!=akhir)
126       {
127         b1=b1->next;
128       }
129       hapus=akhir;
130       b1->next=NULL;
131       akhir=b1;
132     }
133 }
```

Gambar 1.2 `hapus_akhir`

```
void hapus_tengah(int x)
{
    tsimpul *b;
   int ada=0;
   b=awal;
   if(awal==NULL)
        cout<<"Maaf....List Kosong!!!\n";
   else if(b->info==x)
        hapus_awal();
   else if(akhir->info==x)
    hapus_akhir();
   else
   {
    while(b!=NULL)
    {
        if(b->info==x)
         {
            ada++;
         }
            b=b->next;
    }
     if(ada==0)
            cout<<"\n-->Data Yang Di Masukkan Tidak Valid<--\n\n";
     else
     {
     b=awal;
     while(b->next->info!=x)
     {
        b=b->next;
     }
    b1=b->next;
      b2=b1->next;
      hapus=b1;
      b->next=b2;
      }
   }
}
```

Gambar 1.3 `hapus_tengah`

Tabel 1.1 Perbandingan Delete

Operasi	Pointer berubah	Cost
Kiri	`awal` (+ `akhir` jika 1 node)	$O(1)$
Kanan	`b1->next`, `akhir`	$O(n)$
Tengah	`b->next` (+ `akhir` jika target terakhir)	$O(n)$

1.2 Stack

Stack adalah struktur data LIFO (*Last In First Out*): data yang masuk terakhir keluar pertama (Wikipedia, 2026b). Di program ini stack dibuat dari linked list. `atas` jadi puncak (top) dan `jumlah` membatasi maksimal 10 data. Operasi utamanya:

- **Push** (`tambah_stack`): kalau `jumlah == 10`, stack penuh. Kalau belum, buat node baru, sambungkan `data->next = atas`, baru `atas = data`. Urutannya penting, kalau `atas` diubah duluan node lama hilang.
- **Pop** (`hapus_stack`): kalau `atas == NULL`, stack kosong. Kalau tidak, simpan `atas` ke `hapus`, geser `atas = atas->next`, lalu `jumlah--`.
- **Hapus semua** (`hapus_semua`): pop diulang sampai `atas == NULL`, lalu `jumlah = 0`.

Push dan pop cuma menyentuh `atas`, jadi keduanya $O(1)$.

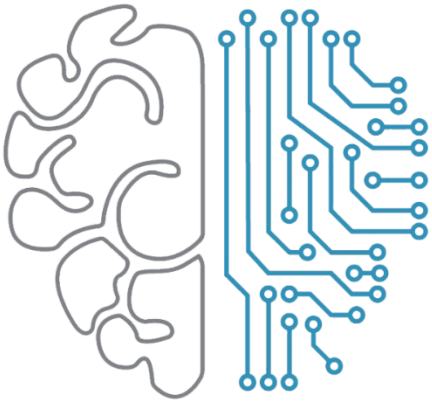
```
void tambah_stack(int x)
{
    if(jumlah==10)
    {
        cout<<"Data Penuh!!!\n";
    }
    else
    {
     data=new simpul;
       data->info=x;
       data->next=atas;
       atas=data;
       jumlah++;
    }
}
```

Gambar 1.4 `tambah_stack` (push)

```
void hapus_stack()
{
    if(atas==NULL)
    {
        cout<<"Maaf....Data Kosong!!!\n";
    }
    else
    {
       hapus=atas;
       atas=atas->next;
       jumlah--;
       cout<<"Penghapusan Berhasil\n";
    }
}
```

Gambar 1.5 `hapus_stack` (pop)

LAPORAN PRAKTIKUM
STRUKTUR DATA



NAMA : ZAIN AKBAR RIZKIA

NIM : 202531091

KELAS : E

DOSEN : Rahma Farahningrum, S.Kom., M.Kom.

NO. PC : 4

ASISTEN : 1. Todo Christanto Pangamudi Simanjuntak

: 2. Harits Nakhlah Putra

: 3. Muhammad Fadel Desyaputra

: 4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026

BAB II

LAPORAN PRAKTIKUM

2.1 Source Code Lengkap

Source code asli dipotong dua, lalu digabung horizontal. Bagian kiri baris 1–111, bagian kanan baris 112–221.

```
#include <conio.h>
#include <iostream.h>
#include <stdio.h>

typedef struct simpul tsimpul;
struct simpul
{
    int info;
   tsimpul *next;
};
tsimpul *awal = NULL, *akhir = NULL, *data, *hapus, *b1, *b2;

void muncul()
{
    b1=awal;
   cout<<"\n       DATA ==>  ";
   while(b1!=NULL)
   {
    cout<<b1->info;
      b1=b1->next;
      cout<<"  ";
   }
}

void tambah_awal(int x)
{
   data=new simpul;
   data->info=x;
   data->next=NULL;
   if(awal==NULL)
   {
    awal=akhir=data;
   }
   else
   {
    data->next=awal;
      awal=data;
   }
}

void tambah_akhir(int x)
{
   data=new simpul;
   data->info=x;
   data->next=NULL;
   if(awal==NULL)
   {
        awal=akhir=data;
   }
   else
   {
    akhir->next=data;
      akhir=data;
   }
}

void tambah_tengah(int in,int setelah)
{
    b1=awal;
   int ada=0;
   while (b1 != NULL)
   {
        if (b1->info == setelah)
        {
            ada++;
        }
        b1 = b1->next;
   }
   if(ada==0)
    cout<<"\n-->Data Yang Di Masukkan Tidak valid<--\n\n";
   else
   {
    b1=awal;
      b2=b1->next;
    while(b1->info!=setelah)
    {
        b1=b1->next;
        b2=b1->next;
    }
        data=new simpul;
    data->info=in;
      b1->next=data;
      if(b1==akhir)
      {
         akhir=data;
      }
      data->next=b2;
   }
}

void hapus_awal()
{
    if(awal==NULL)
   {
        cout<<"Maaf....List Kosong!!!\n";
   }
   else if (awal==akhir)
   {
    hapus=awal;
    awal=NULL;
    akhir=NULL;
   }
   else
   {
      hapus=awal;
      awal=awal->next;
   }
}

void hapus_akhir()
{
```

```
   if(awal==NULL)
   {
        cout<<"Maaf....List Kosong!!!\n";
   }
    else if(awal==akhir)
    {
    hapus=awal;
    awal=NULL;
    akhir=NULL;
   }
   else
   {
      b1=awal;
     while(b1->next!=akhir)
      {
        b1=b1->next;
      }
      hapus=akhir;
      b1->next=NULL;
      akhir=b1;
   }
}

void hapus_tengah(int x)
{
    tsimpul *b;
   int ada=0;
   b=awal;
   if(awal==NULL)
        cout<<"Maaf....List Kosong!!!\n";
   else if(b->info==x)
        hapus_awal();
   else if(akhir->info==x)
    hapus_akhir();
   else
   {
    while(b!=NULL)
    {
        if(b->info==x)
        {
            ada++;
        }
            b=b->next;
    }
      if(ada==0)
            cout<<"\n-->Data Yang Di Masukkan Tidak Valid<--\n\n";
      else
      {
      b=awal;
      while(b->next->info!=x)
      {
        b=b->next;
      }
    b1=b->next;
      b2=b1->next;
      hapus=b1;
      b->next=b2;
      }
   }
}

void main()
{
   int pilih,n,nn;
   char lagi;
   do
   {
      clrscr();
      cout<<"=============================================\n";
      cout<<"||         PROGRAM SINGLE LINKED LIST        ||\n";
      cout<<"=============================================\n";
      muncul();
      cout<<"\n\n1. Tambah Awal";
      cout<<"\n2. Tambah Akhir";
      cout<<"\n3. Tambah Tengah";
      cout<<"\n4. Hapus Awal";
      cout<<"\n5. Hapus Akhir";
      cout<<"\n6. Hapus Tengah\n=========================";
      cout<<"\nPilihan Anda : ";cin>>pilih;
      cout<<"=========================\n";
      switch(pilih)
    {
        case 1:
          cout<<"Masukkan Data    : ";cin>>n;
          tambah_awal(n);
          break;
        case 2:
          cout<<"Masukkan Data    : ";cin>>n;
          tambah_akhir(n);
          break;
        case 3:
          cout<<"Masukkan Data    : ";cin>>n;
          cout<<"Dimasukkan Setelah : ";cin>>nn;
          tambah_tengah(n,nn);
          break;
        case 4:
          hapus_awal();
          break;
          case 5:
          hapus_akhir();
          break;
          case 6:
          cout<<"data yg ingin di hapus : ";cin>>nn;
          hapus_tengah(nn);
          break;
    }
    cout<<"Lagi (y/t)? ";cin>>lagi;
   }
   while(lagi=='y'||lagi=='Y');
}
```

Gambar 2.1 Source Code Single Linked List (`01-linkedlist.cpp`)

```cpp
#include <conio.h>
#include <iostream.h>
#include <stdio.h>

typedef struct simpul tsimpul;
struct simpul
{
    int info;
   tsimpul *next;
};
tsimpul *atas = NULL, *data, *hapus, *b1;
int jumlah=0;

void muncul()
{
    b1=atas;
   cout<<"\n        STACK ==>  ";
   if(atas==NULL)
   {
    cout<<"KOSONG";
   }
   while(b1!=NULL)
   {
    cout<<b1->info;
      b1=b1->next;
      cout<<"  ";
   }
   cout<<"\n        Jumlah Data : "<<jumlah;
}

void tambah_stack(int x)
{
   if(jumlah==10)
   {
        cout<<"Data Penuh!!!\n";
   }
   else
   {
    data=new simpul;
      data->info=x;
      data->next=atas;
      atas=data;
      jumlah++;
   }
}

void hapus_stack()
{
    if(atas==NULL)
   {
        cout<<"Maaf....Data Kosong!!!\n";
   }
   else
   {
      hapus=atas;
      atas=atas->next;
      jumlah--;
      cout<<"Penghapusan Berhasil\n";
   }
}

void hapus_semua()
{
    if(atas==NULL)
    {
        cout<<"Maaf....Data Kosong!!!\n";
   }
    else
    {
      while(atas!=NULL)
      {
        hapus=atas;
        atas=atas->next;
      }
      jumlah=0;
      cout<<"Semua Data Terhapus\n";
   }
}

void main()
{
   int pilih,n;
   char lagi;
   do
   {
      clrscr();
      cout<<"=============================================\n";
      cout<<"||              PROGRAM STACK              ||\n";
      cout<<"=============================================\n";
      muncul();
      cout<<"\n\n1. Tambah Data STACK";
      cout<<"\n2. Hapus Data Teratas";
      cout<<"\n3. Hapus Isi STACK";
      cout<<"\n4. Keluar Program\n=========================";
      cout<<"\nPilihan Anda : ";cin>>pilih;
      cout<<"=========================\n";
      switch(pilih)
    {
        case 1:
          cout<<"Masukkan Data    : ";cin>>n;
          tambah_stack(n);
          break;
        case 2:
          hapus_stack();
          break;
         case 3:
         hapus_semua();
         break;
         case 4:
         cout<<"\n SELESAI \n";
         getch();
         return;
    }
     cout<<"Lagi (y/t)? ";cin>>lagi;
   }
   while(lagi=='y'||lagi=='Y');
}
```

Gambar 2.2 Source Code Stack (`02-stack.cpp`)

2.2 Alur Kerja Delete Kiri, Delete Kanan, dan Delete Tengah

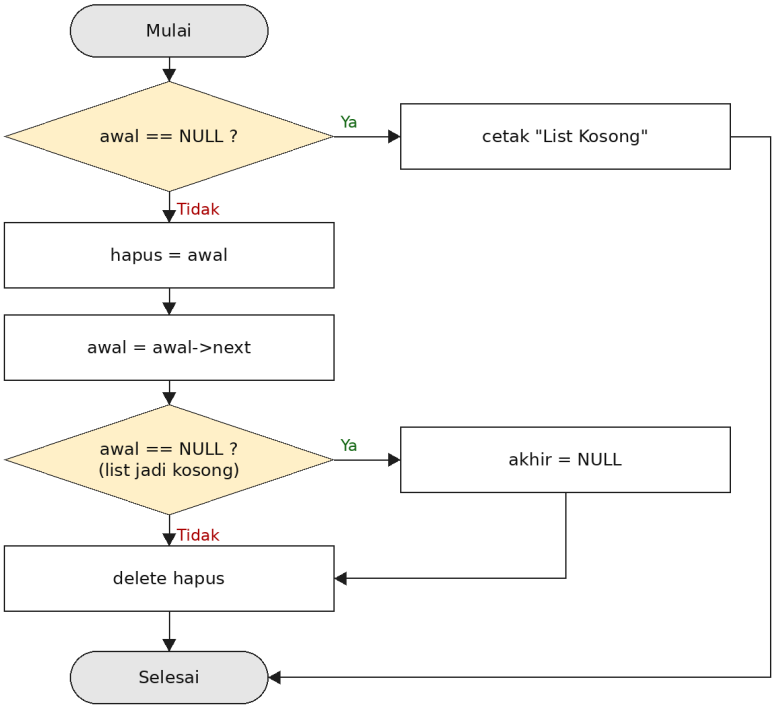
Semua fungsi memakai variabel global dari struct `simpul` di baris 5–11: `awal`, `akhir`, `hapus` (node yang akan dibuang), serta `b1` dan `b2` (pointer bantu).

2.2.1 Delete Kiri

```cpp
void hapus_awal()
{
    if(awal==NULL)
   {
        cout<<"Maaf....List Kosong!!!\n";
   }
   else if (awal==akhir)
   {
    hapus=awal;
    awal=NULL;
    akhir=NULL;
   }
   else
   {
      hapus=awal;
      awal=awal->next;
   }
}
```

Gambar 2.3 Kode `hapus_awal` Asli

Baris 93 mengecek list kosong. Baris 97 menangani list dengan satu node: `awal` dan `akhir` di-`NULL`-kan. Kasus umum ada di baris 103–106: `hapus` menyimpan node depan, lalu `awal` pindah ke `awal->next`. Tidak ada loop, jadi satu langkah saja.



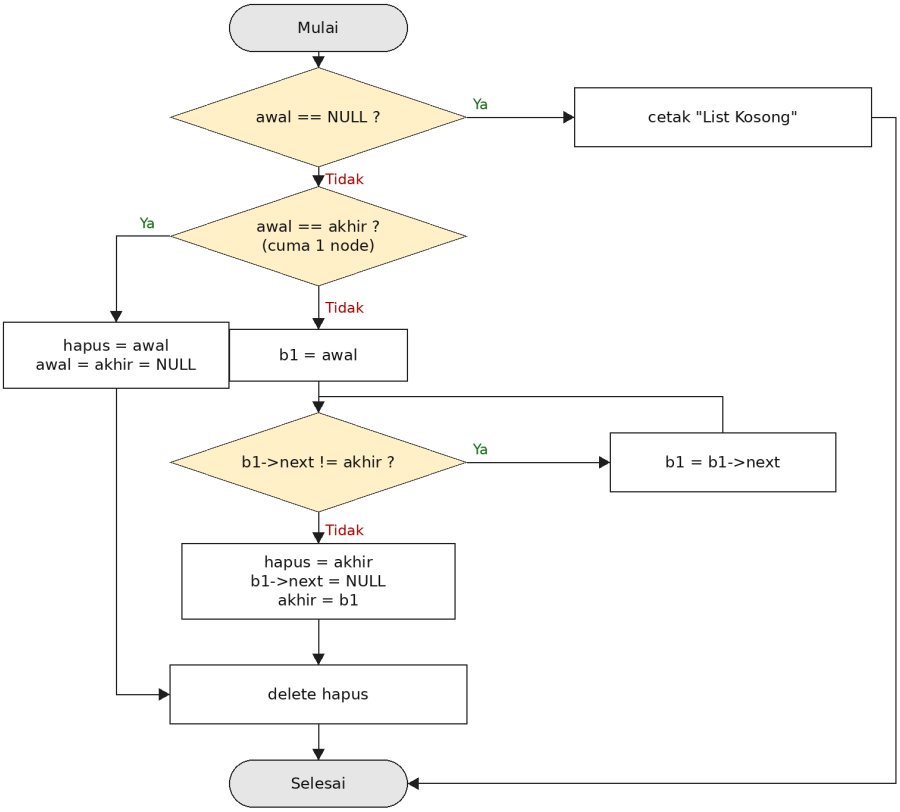
Gambar 2.4 Diagram Alur Delete Kiri

2.2.2 Delete Kanan

```
110 void hapus_akhir()
111 {
112     if(awal==NULL)
113    {
114         cout<<"Maaf....List Kosong!!!\n";
115    }
116     else if(awal==akhir)
117     {
118    hapus=awal;
119     awal=NULL;
120     akhir=NULL;
121    }
122    else
123    {
124         b1=awal;
125       while(b1->next!=akhir)
126       {
127         b1=b1->next;
128       }
129       hapus=akhir;
130       b1->next=NULL;
131       akhir=b1;
132    }
133 }
```

Gambar 2.5 Kode `hapus_akhir` Asli

Kasus satu node sama seperti delete kiri (baris 116–121). Kasus umum: `b1` mulai dari `awal` lalu maju selama `b1->next != akhir` (baris 124–128), jadi berhenti di node sebelum terakhir. Baris 129–131: `hapus` menunjuk node terakhir, `b1->next = NULL` memutus sambungan, dan `akhir = b1` memindahkan tail. Loop inilah yang bikin delete kanan lebih lambat.

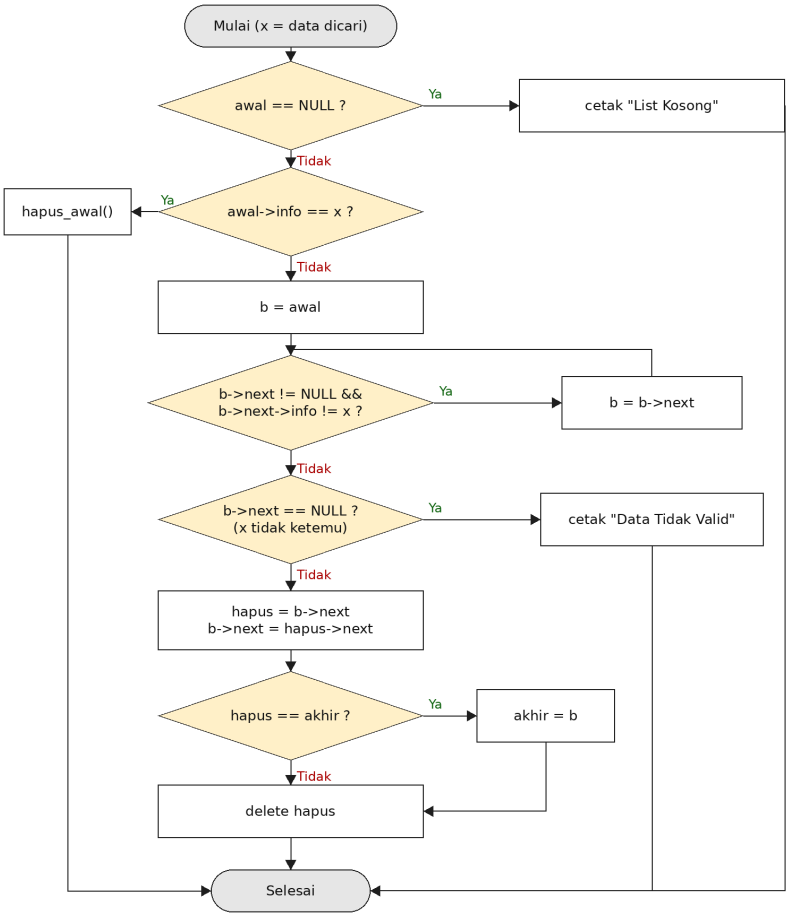


Gambar 2.6 Diagram Alur Delete Kanan

2.2.3 Delete Tengah

```
void hapus_tengah(int x)
{
    tsimpul *b;
   int ada=0;
   b=awal;
   if(awal==NULL)
        cout<<"Maaf....List Kosong!!!\n";
   else if(b->info==x)
        hapus_awal();
   else if(akhir->info==x)
    hapus_akhir();
   else
   {
    while(b!=NULL)
    {
        if(b->info==x)
         {
            ada++;
         }
            b=b->next;
    }
      if(ada==0)
            cout<<"\n-->Data Yang Di Masukkan Tidak Valid<--\n\n";
      else
      {
      b=awal;
      while(b->next->info!=x)
      {
         b=b->next;
      }
     b1=b->next;
       b2=b1->next;
       hapus=b1;
       b->next=b2;
       }
   }
}
```

Gambar 2.7 Kode `hapus_tengah` Asli



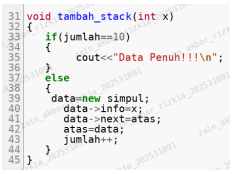
Gambar 2.8 Diagram Alur Delete Tengah

Baris 142 dan 144 mengalihkan ke `hapus_awal()` / `hapus_akhir()` kalau `x` ada di ujung. Baris 148–157 menghitung dulu apakah `x` ada lewat variabel `ada`, kalau 0 muncul pesan tidak valid. Baris 160–164 mencari `b`, yaitu node sebelum `x`. Lalu baris 165–168: `b1` = target, `b2` = node sesudahnya, dan `b->next = b2` melewati target.

2.3 Alur Kerja Stack

Stack memakai struct yang sama, tetapi cuma satu pointer utama `atas`, ditambah penghitung `jumlah` (baris 5–12).

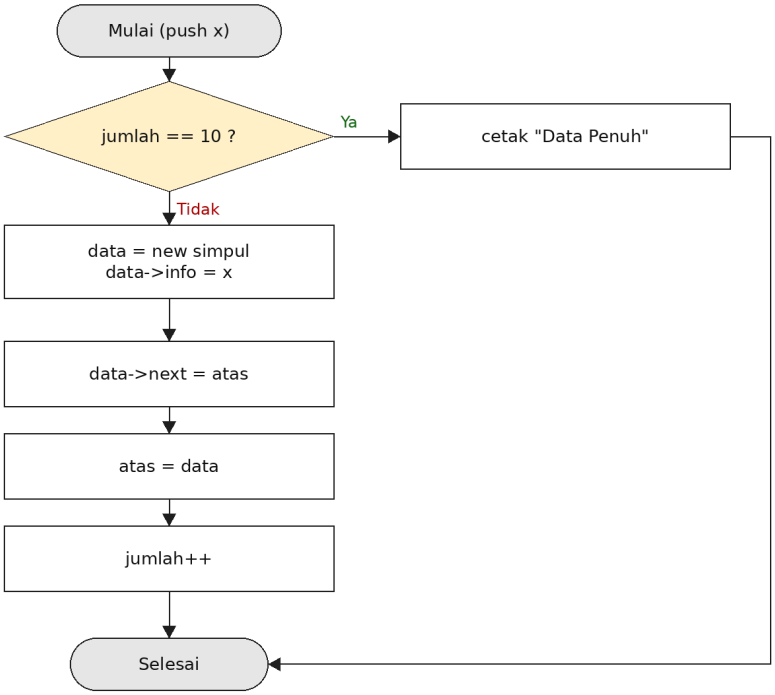
2.3.1 Push



```
31 void tambah_stack(int x)
32 {
33     if(jumlah==10)
34     {
35         cout<<"Data Penuh!!!\n";
36     }
37     else
38     {
39      data=new simpul;
40         data->info=x;
41         data->next=atas;
42         atas=data;
43         jumlah++;
44     }
45 }
```

Gambar 2.9 Kode `tambah_stack` Asli

Baris 33 menolak data baru kalau `jumlah == 10`. Baris 39–40 membuat node dan mengisi `info`. Baris 41 menyambungkan node baru ke stack lama (`data->next = atas`), baru baris 42 memindahkan `atas`. Baris 43 menambah `jumlah`.



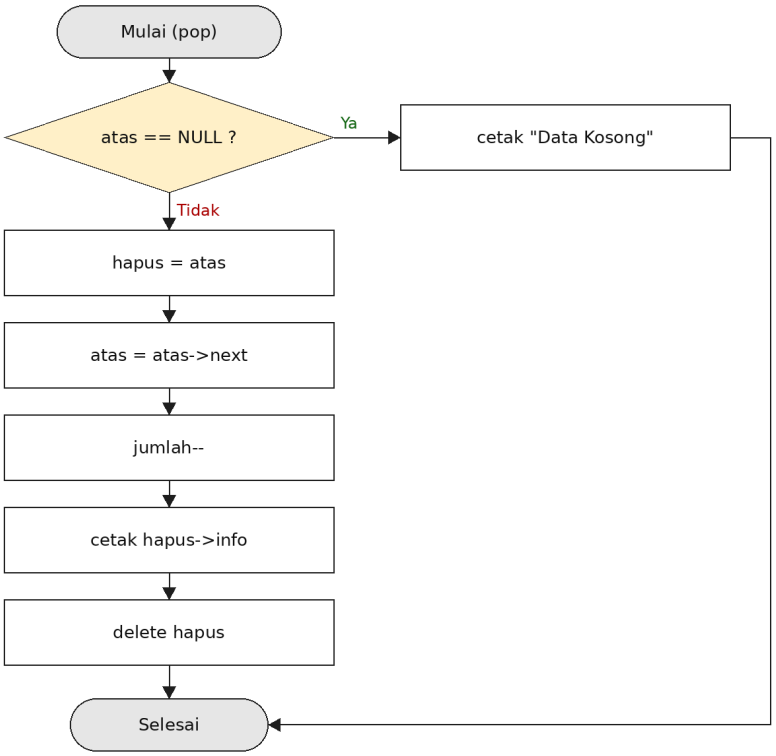
Gambar 2.10 Diagram Alur Push

2.3.2 Pop

```
47 void hapus_stack()
48 {
49     if(atas==NULL)
50     {
51         cout<<"Maaf....Data Kosong!!!\n";
52     }
53     else
54     {
55         hapus=atas;
56         atas=atas->next;
57         jumlah--;
58         cout<<"Penghapusan Berhasil\n";
59     }
60 }
```

Gambar 2.11 Kode `hapus_stack` Asli

Baris 49 mengecek stack kosong. Baris 55–57: `hapus` menyimpan puncak, `atas` turun ke node bawahnya, `jumlah` dikurangi.



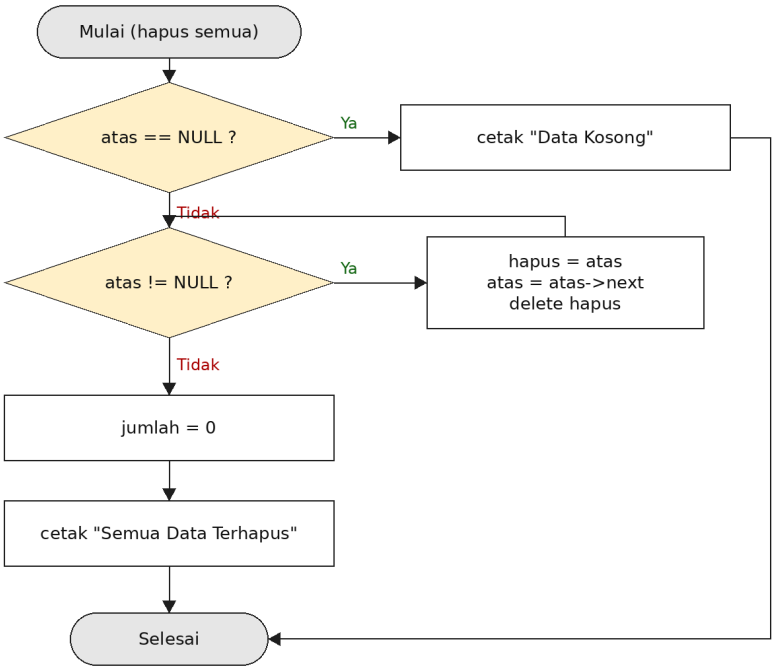
Gambar 2.12 Diagram Alur Pop

2.3.3 Hapus Semua

```
62 void hapus_semua()
63 {
64     if(atas==NULL)
65     {
66         cout<<"Maaf....Data Kosong!!!\n";
67     }
68     else
69     {
70       while(atas!=NULL)
71       {
72         hapus=atas;
73         atas=atas->next;
74       }
75       jumlah=0;
76       cout<<"Semua Data Terhapus\n";
77     }
78 }
```

Gambar 2.13 Kode `hapus_semua` Asli

Loop di baris 70–74 mengulang pop sampai `atas == NULL`, lalu baris 75 mereset `jumlah = 0`.



Gambar 2.14 Diagram Alur Hapus Semua

2.4 Perbaikan Program

Perubahan dicatat dengan git: `git diff asli syntax` untuk syntax dan `git diff syntax fix` untuk logika. Merah = baris dihapus, hijau = baris ditambah. Hasilnya dikompilasi dengan `g++ -std=c++17 -Wall -Wextra -pedantic` tanpa error maupun warning, lalu diuji dengan AddressSanitizer.

2.4.1 Syntax Standar C++

Source asli ditulis untuk Turbo C++, jadi banyak syntax yang ditolak compiler modern:

- `#include <iostream.h>` diganti `<iostream>`, karena header lama sudah tidak ada. `<conio.h>` dan `<stdio.h>` dibuang karena tidak dipakai lagi.
- `cout` dan `cin` dipakai lewat `using std::cout;` dan `using std::cin;`, bukan `using namespace std;`. Alasannya variabel global `data` bentrok dengan `std::data` di C++17.
- `void main()` diganti `int main()` dan diakhiri `return 0;`, sebab standar C++ mewajibkan `main` mengembalikan `int`.
- `clrscr()` (conio) diganti `system(CLS)`, dengan `CLS` bernilai `"cls"` di Windows dan `"clear"` di Linux/macOS. `getch()` pada stack diganti `cin.ignore(); cin.get();`.

```
--- a/src/01-linkedlist.cpp
+++ b/src/01-linkedlist.cpp
@@ -1,4 +1,11 @@
-#include <conio.h>
-#include <iostream.h>
-#include <stdio.h>
+#include <iostream>
+#include <cstdlib>
+using std::cout;
+using std::cin;
+
+#ifdef _WIN32
+#define CLS "cls"
+#else
+#define CLS "clear"
+#endif

@@ -172,3 +179,3 @@ void hapus_tengah(int x)

-void main()
+int main()
 {
@@ -178,3 +185,3 @@ void main()
     {
-    clrscr();
+      system(CLS);
       cout<<"=============================================\n";
@@ -220,2 +227,3 @@ void main()
    while(lagi=='y'||lagi=='Y');
+   return 0;
 }
```

Gambar 2.15 `git diff` Syntax Linked List

```
--- a/src/02-stack.cpp
+++ b/src/02-stack.cpp
@@ -1,4 +1,11 @@
-#include <conio.h>
-#include <iostream.h>
-#include <stdio.h>
+#include <iostream>
+#include <cstdlib>
+using std::cout;
+using std::cin;
+
+#ifdef _WIN32
+#define CLS "cls"
+#else
+#define CLS "clear"
+#endif

@@ -79,3 +86,3 @@ void hapus_semua()

-void main()
+int main()
 {
@@ -85,3 +92,3 @@ void main()
     {
-    clrscr();
+      system(CLS);
       cout<<"=============================================\n";
@@ -110,4 +117,5 @@ void main()
         cout<<"\n SELESAI \n";
-        getch();
-        return;
+        cin.ignore();
+        cin.get();
+        return 0;
     }
@@ -116,2 +124,3 @@ void main()
   while(lagi=='y'||lagi=='Y');
+  return 0;
 }
```

Gambar 2.16 `git diff` Syntax Stack

2.4.2 Logika Delete Kiri, Delete Kanan, dan Delete Tengah

```
--- a/src/01-linkedlist.cpp
+++ b/src/01-linkedlist.cpp
@@ -103,8 +103,2 @@ void hapus_awal()
     }
-    else if (awal==akhir)
-    {
-     hapus=awal;
-     awal=NULL;
-     akhir=NULL;
-    }
     else
@@ -113,2 +107,9 @@ void hapus_awal()
        awal=awal->next;
+       if(awal==NULL)
+       {
+         akhir=NULL;
+       }
+       delete hapus;
+       hapus=NULL;
+       cout<<"Hapus Awal Berhasil\n";
     }
@@ -127,2 +128,5 @@ void hapus_akhir()
      akhir=NULL;
+     delete hapus;
+     hapus=NULL;
+    cout<<"Hapus Akhir Berhasil\n";
     }
@@ -138,2 +142,5 @@ void hapus_akhir()
       akhir=b1;
+      delete hapus;
+      hapus=NULL;
+      cout<<"Hapus Akhir Berhasil\n";
     }
```

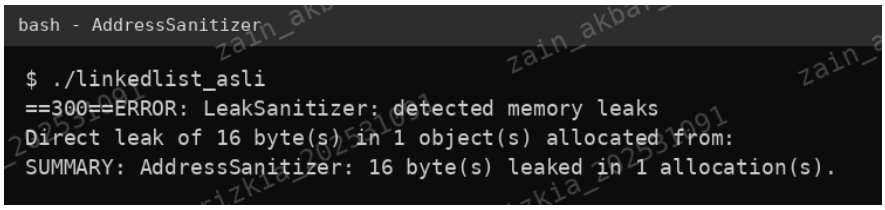
```
@@ -144,26 +151,10 @@ void hapus_tengah(int x)
     tsimpul *b;
-    int ada=0;
-    b=awal;
     if(awal==NULL)
         cout<<"Maaf....List Kosong!!!\n";
-    else if(b->info==x)
+    else if(awal->info==x)
         hapus_awal();
-    else if(akhir->info==x)
-     hapus_akhir();
     else
     {
-     while(b!=NULL)
-     {
-        if(b->info==x)
-         {
-            ada++;
-         }
-            b=b->next;
-     }
-      if(ada==0)
-            cout<<"\n-->Data Yang Di Masukkan Tidak Valid<--\n\n";
-      else
-      {
       b=awal;
-      while(b->next->info!=x)
+      while(b->next!=NULL && b->next->info!=x)
       {
@@ -171,6 +162,15 @@ void hapus_tengah(int x)
       }
-    b1=b->next;
-    b2=b1->next;
-    hapus=b1;
-    b->next=b2;
+      if(b->next==NULL)
+            cout<<"\n-->Data Yang Di Masukkan Tidak Valid<--\n\n";
+      else
+      {
+        hapus=b->next;
+        b->next=hapus->next;
+        if(hapus==akhir)
+        {
+            akhir=b;
+        }
+        delete hapus;
+        hapus=NULL;
+        cout<<"Hapus Tengah Berhasil\n";
       }
```

Gambar 2.17 `git diff` Single Linked List (dipotong dua, digabung horizontal)

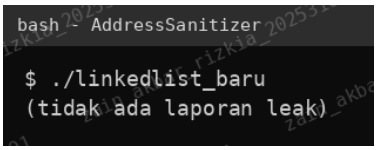
Masalah di versi asli dan perbaikannya:

- **Node tidak pernah di-`delete`.** Ketiga fungsi cuma melepas node dari list, memorinya tetap terpakai (memory leak). Sekarang tiap fungsi diakhiri `delete hapus; hapus = NULL;`. Buktinya, AddressSanitizer melaporkan leak di versi asli dan bersih di versi baru.
- **Tidak ada pesan sukses.** Ditambah `Hapus Awal/Akhir/Tengah Berhasil` supaya hasilnya kelihatan.
- **`hapus_awal` lebih pendek.** Cabang `awal == akhir` digabung ke satu pengecekan `if(awal==NULL) akhir=NULL` setelah `awal` bergeser.

- **`hapus_tengah` disederhanakan.** Dua loop (hitung `ada`, lalu cari) jadi satu loop `b->next != NULL && b->next->info != x`. Variabel `ada` dan `b2` dibuang, dan `akhir = b` di-update kalau yang terhapus node terakhir. Di versi asli, list `1 2 3 2` dihapus `2` menghasilkan `1 2 3` (yang terhapus kemunculan terakhir karena cek `akhir->info` duluan), sekarang menghasilkan `1 3 2` (kemunculan pertama).



Gambar 2.18 Versi Asli (ada leak)



Gambar 2.19 Versi Baru (bersih)

2.4.3 Logika Stack

```
--- a/src/02-stack.cpp
+++ b/src/02-stack.cpp
@@ -64,3 +64,5 @@ void hapus_stack()
       jumlah--;
-      cout<<"Penghapusan Berhasil\n";
+      cout<<"Data "<<hapus->info<<" Berhasil Dihapus\n";
+      delete hapus;
+      hapus=NULL;
    }
@@ -80,3 +82,5 @@ void hapus_semua()
         atas=atas->next;
+        delete hapus;
       }
+      hapus=NULL;
       jumlah=0;
```

Gambar 2.20 `git diff` Stack

Di `hapus_stack` dan `hapus_semua` versi asli juga tidak ada `delete`. Sekarang node dibebaskan, dan `hapus_stack` menampilkan nilai yang dibuang (`Data 30 Berhasil Dihapus`), bukan cuma "Penghapusan Berhasil".

2.5 Hasil Eksekusi

Tiap screenshot menampilkan satu layar program setelah operasi dijalankan.



Gambar 2.21 Delete Tengah (data 3)



Gambar 2.22 Delete Tengah, Data Tidak Ada



Gambar 2.23 Delete Kanan



Gambar 2.24 Delete Kiri



Gambar 2.25 Pop



Gambar 2.26 Hapus Semua



Gambar 2.27 Pop Saat Kosong

DAFTAR PUSTAKA

cppreference.com. (2026). *delete expression*. https://en.cppreference.com/w/cpp/language/delete

Wikipedia. (2026a). *Linked list*. https://en.wikipedia.org/wiki/Linked_list

Wikipedia. (2026b). *Stack (abstract data type)*. https://en.wikipedia.org/wiki/Stack_(abstract_data_type)